OPERATING SYSTEM ORANGE ASSIGNMENT

NAME:TEJAS P

SRN:PES1UG24CS497

SECTION: I

PHASE 1:

1A

```
tejas@tejas-Dell:~/Desktop/PES1UG24CS497-pes-vcs/PES1UG24CS497-pes-vcs$ ./test_o
bjects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
```

1B

```
tejas@tejas-Dell:~/Desktop/PES1UG24CS497-pes-vcs/PES1UG24CS497-pes-vcs$ ./pes in
it
find .pes/objects -type f
bash: ./pes: No such file or directory
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
tejas@tejas-Dell:~/Desktop/PES1UG24CS497-pes-vcs/PES1UG24CS497-pes-vcs$
```

PHASE 2:

2A

```
tejas@tejas-Dell:~/Desktop/PES1UG24CS497-pes-vcs/PES1UG24CS497-pes-vcs$ git add
tree.c
git commit -m "Phase 2: fix tree_from_index stub, remove premature index depende
ncy"
make test_tree
[main 6481ba7] Phase 2: fix tree_from_index stub, remove premature index depende
ncy
 1 file changed, 3 insertions(+), 74 deletions(-)
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
```

2B

```
tejas@tejas-Dell:~/Desktop/PES1UG24CS497-pes-vcs/PES1UG24CS497-pes-vcs$ ./test_t
ree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

PHASE 3:

3A

```
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_sequence.sh
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  .git
  untracked:  object.c
  untracked:  count++
  untracked:  index.h
  untracked:  README.md
  untracked:  Makefile
  untracked:  commit.h
  untracked:  count
  untracked:  mode
  untracked:  tree.c
  untracked:  name[name_len]
  untracked:  index.c
  untracked:  hash
  untracked:  entries[tree_out-
  untracked:  count]
  untracked:  pes.h
  untracked:  commit.c
  untracked:  test_tree.c
  untracked:  tree.h
```

3B

```
tejas@tejas-Dell:~/Desktop/PES1UG24CS497-pes-vcs/PES1UG24CS497-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776681122 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776681122 6 file2.txt
```

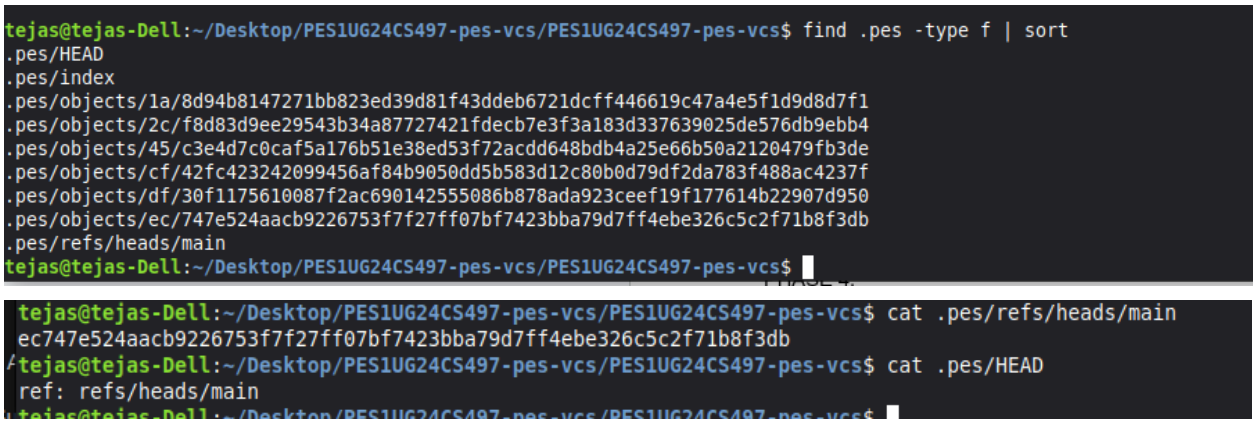
PHASE 4:

4A

```
tejas@tejas-Dell:~/Desktop/PES1UG24CS497-pes-vcs/PES1UG24CS497-pes-vcs$ ./pes log
commit ec747e524aacb9226753f7f27ff07bf7423bba79d7ff4ebe326c5c2f71b8f3db
Author: Tejas P PES1UG24CS497>
Date:   1776681826

    third commit
```

4B



Phase 5&6:(question answers)

Phase 5

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans:Implementing pes checkout basically means you have to update the HEAD file so it points to the new branch ref, like ref: refs/heads/main. The real headache is syncing the working directory; you have to delete files that aren't in the new branch, update the ones that changed, and add the new ones, all

while making sure you don't accidentally overwrite a user's unsaved local work.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.
Ans:To catch a "dirty" directory conflict, I'd just compare the mtime and size of files in the working directory against what's in the current index. If those don't match, the file is modified. Then, I'd check if that file's hash in my current index is different from the hash in the target branch's tree. If both are true, it's a conflict, and I'd have to block the checkout so the user doesn't lose their data.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans:When you’re in a "Detached HEAD," it just means HEAD has a raw commit hash in it instead of a branch name. You can still make commits, but they aren't attached to any branch, so they become "orphans" the moment you switch away to something else. To get them back, you’d have to find that specific hash in the object store and manually create a new branch ref that points to it.

Phase 6
Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.
Ans:To clean up the object store, I’d use a mark-and-sweep algorithm. First, I'd start at the "roots"—all the branch hashes and HEAD—and recursively follow every tree and parent pointer to "mark" every object that is still reachable. For a huge repo with 100k commits, I’d probably end up visiting

a few hundred thousand objects at least, including all the trees and blobs linked to those commits. Finally, I'd just "sweep" through the .pes/objects folder and delete anything that wasn't marked.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?
Ans:Running GC while someone is committing is super risky because of race conditions. For example, a new commit might calculate a hash for a file and assume it's already in the store, but the GC might see that same object as "unreachable" at that exact second and delete it before the commit finishes. Real Git fixes this by having a grace period, where the GC won't touch any unreachable object unless it's at least two weeks old, giving new operations plenty of time to finish.